HackIndia Hub

Project Documentation

A one-stop website to discover hackathons across India, find teammates, and never miss a deadline.

What this document covers	
Purpose	Explain the project to friends in simple language
File documented	index.html (the main page of the website)
Also explained	Every technology / technique used, and exactly how it is used

Note: Only `index.html` was shared. The CSS and JavaScript files it loads (`css/*.css`, `js/*.js`) were not included, so this document explains what the HTML page contains and what it expects those files to do. Where I describe what a JS file does, it is inferred from the comments and function names in the HTML.

1. What is HackIndia Hub?

HackIndia Hub is a website for college students in India. Hackathons (coding competitions) are announced on many different places - Devfolio, Unstop, college websites, Smart India Hackathon and more. Students often miss them. This website collects them in **one place** so a student can search, filter, save and get reminded.

Main things a visitor can do

- **Discover** - browse and filter a directory of hackathons (by state, mode, technology, difficulty, prize).
- **Map View** - see hackathons on an interactive map of India.
- **Find Teammates** - post or answer requests like 'I need a React developer for SIH'.
- **For You** - a personal dashboard with saved hackathons and recommendations based on your profile.
- **Admin** - a control room where submitted hackathons are checked, to block fake listings.
- **Post Hackathon** - organizers can submit their own event for verification.
- **AI Assistant** - a chat drawer that suggests hackathons and team ideas.
- **Alerts, Calendar and Share** - notifications, add-to-calendar, and WhatsApp/LinkedIn/Twitter sharing.

2. Technologies Used (Summary)

Technology	What it is	Where you see it in this project
HTML5	The skeleton / structure of a web page	The whole index.html file
CSS3	Colours, layout, fonts, animations	3 linked files: styles.css, components.css, map.css
Vanilla JavaScript	Makes the page interactive (no framework)	10 linked files in js/ folder + inline onclick handlers
Inline SVG	Icons and the India map drawn with code	Logo, search icon, bell, nav icons, map
SEO meta tags	Help Google understand the page	title, description, keywords, author
Open Graph / Twitter Cards	Make a nice preview when link is shared	og:title, og:image, twitter:card ...
Forms + HTML validation	Collect user input safely	Post Hackathon, Profile, Team, Report forms
Modals	Pop-up windows over the page	9 modals (details, forms, share, calendar ...)
Single-Page navigation	Switch screens without reloading	navigateToView('discover' / 'map' / ...)
Responsive design	Works on phone and laptop	Mobile filter bar, bottom navigation bar

3. How the Project Is Organized

The page links to other files. Think of index.html as the **body**, CSS as the **clothes**, and JavaScript as the **brain**.

```
hackindia-hub/
 |-- index.html            <- the page (shared with me)
 |-- assets/hero_banner.jpg
 |-- css/
 |    |-- styles.css       <- global look: colours, fonts, hero, navbar
 |    |-- components.css   <- cards, buttons, modals, forms
 |    '-- map.css          <- styling for the India map
 '-- js/
      |-- data.js          <- the hackathon data
      |-- dashboard.js     <- "For You" page
      |-- teams.js         <- teammate finder
      |-- organizer.js     <- Post Hackathon form logic
      |-- admin.js         <- moderation / admin page
      |-- ai-assistant.js  <- AI chat drawer
      |-- notifications.js <- bell alerts
      |-- calendar-share.js<- calendar + share features
      |-- map.js           <- draws the India map
      '-- app.js           <- main logic: filters, rendering, navigation
```

Why the JavaScript order matters

At the bottom of index.html the scripts load in a fixed order: `data.js` first, `app.js` last. Data must exist before other files use it, and the main app file runs last once everything else is ready. If you change this order the site can break.

```
<script src="js/data.js"></script>      <!-- 1. data first -->
<script src="js/dashboard.js"></script>
...
<script src="js/app.js"></script>       <!-- last: starts the app -->
```

4. The <head> Section - Invisible but Important

Everything inside `<head>` is not shown on the page, but it controls how the page behaves in browsers, Google and WhatsApp.

4.1 SEO tags (Google search)

SEO means Search Engine Optimization. These tags tell Google what the page is about.

```
<title>HackIndia Hub | Discover Hackathons Across India...</title>
<meta name="description" content="Never miss a hackathon again...">
<meta name="keywords" content="hackathons india, college hackathons...">
```

How it is used: The title is the blue link people see in Google. The description is the small text below it. Keywords list the phrases students are likely to search (hackathon hyderabad, smart india hackathon 2026, devfolio ...).

4.2 Viewport tag (mobile friendly)

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without this, phones show a tiny zoomed-out desktop page. This line makes the page fit the phone screen.

4.3 Open Graph and Twitter tags (link previews)

```
<meta property="og:title" content="HackIndia Hub - Never Miss a Hackathon Again">
<meta property="og:image" content="assets/hero_banner.jpg">
<meta name="twitter:card" content="summary_large_image">
```

How it is used: When a friend pastes your link in WhatsApp, LinkedIn or Twitter, these tags create the preview card (big image + title + description). One tip: for real sharing, the `og:image` value should be a full URL like `https://hackindiahub.in/assets/hero_banner.jpg`, because many apps cannot read relative paths.

4.4 Favicon - an emoji as the tab icon

```
<link rel="icon" href="data:image/svg+xml,<svg ...><text ...>rocket emoji</text></svg>">
```

How it is used: Instead of a separate image file, a tiny SVG containing a rocket emoji is written directly in the link (a 'data URI'). No extra file needed.

4.5 Stylesheets

```
<link rel="stylesheet" href="css/styles.css">
<link rel="stylesheet" href="css/components.css">
<link rel="stylesheet" href="css/map.css">
```

Three CSS files are split by purpose, which keeps the code tidy. The CSS uses **CSS variables** such as `var(--cyan)`, `var(--saffron-light)`, `var(--text-muted)`, `var(--radius-md)`. Change a colour once and the whole site updates. The saffron and cyan colours give the Indian-flag-inspired dark theme.

5. The Page Layout, Top to Bottom

The <body> is built with semantic tags, which means each tag name describes its job.

Part	HTML tag / id	What it does
Top bar	<header class='navbar'>	Logo, 5 menu links, bell icon, Post Hackathon button, profile pill
Notification drawer	#notificationDrawer	Drop-down list of alerts inside the header
Hero	<section id='heroSection'>	Big headline, search bar, quick filter chips, action buttons, 4 stat cards
Main content	<main class='container'>	Holds the 4 'views' (Discover, Teams, Dashboard, Admin)
Modals	div.modal-backdrop (x9)	Pop-up windows, hidden until opened
AI assistant	#aiAssistantFab + #aiAssistantDrawer	Floating button and chat panel
Footer	<footer class='footer'>	Links by city, category, organizers
Mobile nav	div.mobile-bottom-nav	5-icon bar fixed at the bottom on phones

Why semantic tags?

`header`, `nav`, `main`, `section`, `aside`, `footer` help Google and screen readers understand the page, instead of using plain `div` everywhere.

A small note

There are two `<main>` tags (one outer, one inside the discovery layout). HTML allows only one visible main per page. It works, but changing the inner one to `<div>` would be cleaner.

6. Navigation: Switching Screens Without Reloading

This is a **Single Page Application** style. There is only one HTML file. The four 'views' are sections stacked in the page, and only one is visible at a time.

```
<section id="discoverView">...</section>
<section id="teamsView" style="display: none;">...</section>
<section id="dashboardView" style="display: none;">...</section>
<section id="adminView" style="display: none;">...</section>
```

How it is used: Clicking a menu link calls `navigateToView('teams')`. That JavaScript function hides all sections and shows only the chosen one, and moves the 'active' highlight on the menu. The page never reloads, so it feels fast.

```
<a href="javascript:void(0)" class="nav-link" data-view="teams"
   onclick="navigateToView('teams')">Find Teammates</a>
```

Two details to notice:

- `href="javascript:void(0)"` stops the link from jumping anywhere; only the onclick runs.
- `data-view="teams"` is a **data attribute**: a custom label that JavaScript reads to know which link is which.

The same function is reused by the logo, the footer, the profile pill, and the mobile bottom bar - write once, use everywhere.

7. Hero Section, Search and Quick Filters

The first thing a visitor sees. It has four jobs:

- **Headline** with a gradient-colored word: `<span class="text-gradient-saffron">Hackathon</span>`.
- **Search bar**: an `<input id="heroSearchInput">` and a Search button that runs `applyFilters()`.
- **Quick chips**: one-click buttons like AI/ML, Web3, Cyber, Beginner. Each calls `quickHeroCategoryFilter('AI/ML')`.
- **Stat cards**: numbers like 140+ hackathons. They have ids (`statCountHackathons`) so JavaScript can update them with real counts.

```
<button class="hero-chip" onclick="quickHeroCategoryFilter('AI/ML')">AI/ML</button>
```

How it is used: The text inside the quotes is passed to the function as a value (called an *argument*). One function serves every chip, so the code is not repeated seven times.

The `autocomplete="off"` on the search box stops the browser showing its own old suggestions over your results.

8. Discover View: Smart Filters

The left sidebar (`<aside id="filtersSidebar">`) lets students narrow the list. Each filter is an HTML form control with an id that JavaScript reads.

Filter	id	Options
Keyword search	quickSearchInput	Free text (e.g. SIH, Python)
Location / State	filterLocationState	All India, Online, Telangana, Karnataka, Maharashtra, Delhi, Tamil Nadu, Kerala, AP, West Bengal, UP, Gujarat, Rajasthan
Event mode	filterMode	Online / Offline / Hybrid
Technology	filterTech	AI/ML, GenAI, Web, App, Cyber, Blockchain, IoT, Cloud, Data Science, Open Innovation
Difficulty	filterDifficulty	Beginner / Intermediate / Advanced
Eligibility	filterEligibility	College, 1st-2nd year friendly, Open to everyone
Minimum prize	filterPrize	Rs. 1 Lakh, 5 Lakh, 10 Lakh, 25 Lakh (values are numbers like 100000)

How it works

1) The user changes a dropdown. 2) `app.js` reads every filter's value. 3) It loops through the hackathon list from `data.js` and keeps only the matching ones. 4) It redraws the cards inside `#hackathonsGridList`, and shows the count in `#discoveryResultCount`.

Good design choice: each dropdown has an 'all' option with `value="all"`, which means 'do not filter by this'. The prize options store plain numbers (100000) in `value` while showing friendly text (Rs. 1,00,000+) to the user, so the code can compare numbers easily.

Sorting and list/map toggle

The sort dropdown (`sortBySelect`) offers Deadline, Prize, Popular and Newest. The List/Map buttons call `switchViewMode('list' | 'map')`, which shows one container and hides the other (the map container starts with `display:none`).

9. Dynamic Content: Empty Containers Filled by JavaScript

A very important idea in this project. Many boxes in the HTML are **empty on purpose**; JavaScript fills them at run time. The author left comments to show who fills each one:

Empty container (id)	Filled by	Shows
hackathonsGridList	app.js	Hackathon cards
indiaMapWrapper	map.js	The India SVG map
teamsGridContainer	teams.js	Teammate request cards
dashboardContentWrapper	dashboard.js	Personal dashboard
adminViewContent	admin.js	Moderation queue
notificationItemsList	notifications.js	Alert list
aiMessagesContainer	ai-assistant.js	Chat messages
waPreviewCardContainer	calendar-share.js	WhatsApp preview card
modalDetail* ids	app.js	Title, prize, dates, tracks, prizes, timeline, FAQs of the chosen hackathon

Why do it this way?

If you wrote 140 hackathon cards by hand in HTML it would be huge and hard to update. Instead, the data lives in one place (`data.js`) and JavaScript builds the cards from it. Add one new hackathon to the data and the card, map, filters and counts all update.

Each file has one job (one file per feature). This is called **separation of concerns** and makes teamwork easier: one friend can work on teams.js while another works on admin.js.

10. Modals (Pop-up Windows)

A modal is a box that appears on top of the page. All 9 modals are already written in the HTML and hidden. Opening one just adds a CSS class called `active`; closing removes it.

```
<!-- open -->
onclick="document.getElementById('postHackathonModal').classList.add('active')"
<!-- close -->
onclick="closeModal('postHackathonModal')"
```

#	Modal id	Purpose
1	hackathonDetailsModal	Full details with 5 tabs, save / calendar / share / register buttons
2	postHackathonModal	Organizer form to submit a hackathon for verification
3	profileEditModal	Student profile (name, college, year, skills, interests, preferred mode)
4	postTeamModal	Create a teammate request
5	joinTeamModal	Send a message to a team creator
6	shareModal	WhatsApp / LinkedIn / Twitter / copy link

#	Modal id	Purpose
7	calendarModal	Google, Outlook, Apple (.ics) calendar
8	reportHackathonModal	Report a fake or wrong listing
9	alertPreferencesModal	Choose which alerts to receive

The details modal has tabs

Overview, Tracks, Prizes, Timeline, FAQs. Each tab button calls `switchModalTab('prizes')`. The matching content div has an id like `modalTab-prizes`, so JavaScript simply builds the id from the name and shows that panel. Using one modal and refilling it with different data is better than creating 140 separate modals.

11. Forms and Validation

Four forms collect user input: Post Hackathon, Edit Profile, Post Team Request and Report. They use **built-in HTML5 validation** - no extra code needed.

HTML feature	Example from the project	What it does
required	<input ... required />	Browser blocks submit if empty
type='email'	orgContactEmail	Checks the text looks like an email
type='url'	orgRegUrl, orgSourceUrl	Checks it is a proper link
type='date'	orgStartDate, orgDeadline	Shows a calendar picker
type='number'	orgPrizePool, postTeamMaxMembers (min 2, max 6)	Only digits; limits
placeholder	e.g. Hyderabad AI Conclave 2026	Grey hint text inside the box
label for=id	<label for='orgCity'>	Clicking the label focuses the input; helps accessibility

```
<form id="postHackathonForm" onsubmit="handleOrganizerSubmit(event)">
```

How it is used: When the user clicks Submit, the browser first checks all rules. If all pass, `handleOrganizerSubmit(event)` runs. Inside it, the JavaScript will call `event.preventDefault()` so the page does not reload, then read the values and send the hackathon into the admin queue.

Trust and safety features

- Organizers must give an **official email** and a **verification source URL** (college website) - this reduces fake events.
- A **Report** modal lets any student flag fake links, expired events, wrong prizes, or impersonation.
- The **Admin** view is the moderation queue where reports and submissions are reviewed.
- The footer shows a 'Verified sources' badge to build trust.

12. Other Features Explained

12.1 Notifications

The bell button calls `toggleNotificationDrawer()`. The red number badge (2) is a `<span class='badge-counter'>` positioned over the bell with CSS. 'Mark Read' clears it. 'Prefs' opens the Alert Preferences modal.

12.2 Calendar

Three options: Google Calendar and Outlook (open a pre-filled link in a new tab) and an **.ICS file** download. ICS is a universal calendar file format that Apple Calendar and most other apps can import.

12.3 Sharing

The Share modal builds a ready-to-send message and buttons for WhatsApp, LinkedIn, Twitter/X and Copy Link. WhatsApp and the others normally work by opening a special URL (for example `https://wa.me/?text=...`) with the message text in it. A read-only `<textarea>` shows the message so the user can see what will be sent.

12.4 Interactive India Map

`map.js` injects an **SVG** into `#indiaMapWrapper`. SVG is a picture made of code, so states can be coloured and clicked. The legend shows three levels of activity: High (15+ hackathons), Active Hub (5-14), Emerging.

12.5 AI Assistant

A floating button (FAB = Floating Action Button) toggles a side drawer. It has three quick-question buttons that call `sendAiMessage('...')` with a ready question, plus a text box where Enter also sends (`onkeydown="if(event.key==='Enter') sendAiMessage()"`). Whether it replies using real AI or pre-written smart rules depends on `ai-assistant.js`, which I have not seen.

12.6 Personalization ('For You')

The profile (skills, interests, year, experience, city, preferred mode) is saved from the Edit Profile modal. The text says 'Our smart engine uses these details' to match hackathons and teammates. The nav pill shows initials 'SK' and the name from the profile.

13. Coding Techniques Used (Simple Explanation)

Technique	Meaning	Example in this project
Inline event handlers	JS attached directly in HTML with onclick	onclick="navigateToView('map')"
Function arguments	Send a value to a function so it can be reused	quickHeroCategoryFilter('Blockchain')
IDs as hooks	Unique names JS uses to find elements	document.getElementById('shareModal')
Data attributes	Custom info stored on an element	data-view='dashboard', data-tab='faqs'
CSS classes as state	Add/remove a class to show or hide	classList.add('active')
CSS variables	Reusable colours and sizes	var(--cyan), var(--radius-md)
Utility + component CSS	Small helper classes plus reusable parts	btn, btn-primary, btn-outline, btn-sm, btn-lg
Emoji as icons	Quick icons with no image files	search, map, team, star emojis in menu
Inline SVG icons	Sharp icons that scale and take text colour	stroke='currentColor'
Mobile-first extras	Separate UI for small screens	mobile-filter-bar, mobile-bottom-nav
Comment banners	Clear section markers in code	TOP NAVIGATION BAR, MODALS SYSTEM ...

About stroke='currentColor'

All SVG icons use `stroke="currentColor"`, which means the icon takes the text colour of its parent. Change the text colour in CSS and the icon colour follows automatically - no need to edit each SVG.

14. How to Run the Project

- Keep the folder structure from Section 3 (index.html, css/, js/, assets/).
- Double-click `index.html` to open in a browser, or use VS Code 'Live Server' for auto-refresh.
- To publish for free: upload the folder to GitHub Pages, Netlify or Vercel.
- No installation, build step or server is required - it is plain HTML, CSS and JavaScript.

15. Suggestions to Improve

- **Social preview:** make og:image a full URL so WhatsApp can show it.
- **One <main>:** change the inner <main class='discovery-content-area'> to a .
- **Accessibility:** use real <button> tags instead of <a href='javascript:void(0)'> for actions, and add aria-labels to icon-only buttons.
- **Move inline styles to CSS:** many style='...' attributes make changes harder.
- **Hard-coded text:** some labels (e.g. 'Hyderabad / Telangana' in alert preferences, name 'Sai') should come from the saved profile.

- **Honest stats:** 140+ hackathons / 350+ colleges are typed in the HTML; make sure they match real data, and the 'Verified sources' claim is true before launch.
- **Real backend later:** a database and login would let profiles, teams and admin approvals persist across devices.

16. Quick Glossary for Friends

Word	Simple meaning
Hackathon	A coding competition, usually 24-48 hours, where teams build a project
HTML	Structure of the page (headings, buttons, boxes)
CSS	Style of the page (colours, spacing, layout)
JavaScript	Behaviour of the page (clicks, filters, pop-ups)
DOM	The page as the browser sees it; JS can read and change it
SVG	Images drawn with code; sharp at any size
Modal	Pop-up box over the page
SEO	Making Google understand and rank your page
Open Graph	Tags that create the preview card when a link is shared
SPA	Single Page App - switching screens without reloading
FAB	Floating Action Button (the round AI button)
ICS	Calendar file format that most calendar apps can import